Adaptive Step Control for RK Methods

Andrea Tribotti, Leonardo Cittadini, Werther Solazzi
Department of Mathematics: Numerical Methods Exam
University of Bologna

March 2026

Introduction

This report presents an implementation and comparison of adaptive step size control strategies for explicit Runge-Kutta (ERK) methods applied to the numerical solution of initial value problems (IVPs).

The study is structured in two phases:

- in the first phase, we test three embedded Runge-Kutta tableaux—ERK4(3), Bogacki-Shampine BS3(2), and Dormand-Prince DP5(4)—on multiple test problems including the Brusselator system and two scalar ODEs.
  An interactive MATLAB application was developed to facilitate parameter exploration, allowing users to select the ODE, the embedded tableau, tolerance settings, and safety factor.

- in the second phase, we focus exclusively on the Brusselator to conduct a detailed comparison between adaptive and non-adaptive strategies. Specifically, we compare the three adaptive methods evaluated in the first phase—BS3(2), ERK4(3), and DP5(4)—against three non-adaptive implementations: a third-order ERK method (ERK3), a standard fourth-order ERK method (ERK4), and a fourth-order ERK method enhanced with Richardson extrapolation.

The automatic step size control mechanism is based on the framework described by Hairer and Nørsett, which compares numerical approximations of different orders to estimate the local truncation error and adjust the step size accordingly.

1 Theoretical Framework

Explicit Runge-Kutta Methods

An explicit Runge-Kutta method of order p with s stages for solving the initial value problem

$$y' = f(t, y(t)), \quad y(t_0) = y_0$$

computes the numerical solution y_{n+1} at $t_{n+1} = t_n + h$ using the formula

$$y_{n+1} = y_n + h \sum_{i=1}^{s} b_i K_i$$

where the K_i are defined by

$$K_i = f\left(t_n + c_i h, y_n + h\sum_{j=1}^{i-1} a_{ij}K_j\right)$$

The coefficients a_{ij}, b_i, and c_i are chosen to satisfy order conditions that ensure the local truncation error behaves as $O(h^{p+1})$.

Richardson Extrapolation

Richardson extrapolation is a classical technique for improving the accuracy of numerical approximations by exploiting the known asymptotic behavior of the truncation error. For a Runge-Kutta method of order p, given the initial value (t_0, y_0), if we compute two steps with step size h to obtain y_2, and one big step with step size $2h$ to obtain w, the errors satisfy

$$y(t_0 + 2h) - y_2 = 2Ch^{p+1} + \mathcal{O}(h^{p+2})$$

$$y(t_0 + 2h) - w = C(2h)^{p+1} + \mathcal{O}(h^{p+2})$$

where C depends on higher-order derivatives of the solution. Ignoring the terms $\mathcal{O}(h^{p+2})$, the previous formulas allow us to eliminate the constant C and to extrapolate a better value $\hat{y}_2$ for $y(t_0 + 2h)$, for which we obtain

$$y(t_0 + 2h) - \hat{y}_2 \approx \mathcal{O}(h^{p+2})$$

where

$$\hat{y}_2 = y_2 + \frac{y_2 - w}{2^p - 1}$$

In this way, starting from a formula of order p, we obtain an approximation of higher order.

Embedded Runge-Kutta Formulas

An alternative to Richardson extrapolation is to use embedded Runge-Kutta formulas, where two approximations y_1 and $\hat{y}_1$ of orders p and $\hat{p}$ (typically $\hat{p} = p - 1$ or $\hat{p} = p + 1$) share the same function evaluations. The method is characterized by a Butcher tableau with two sets of weights:

$$\begin{array}{c|cccc}
0 & & & & \\
c_2 & a_{21} & & & \\
c_3 & a_{31} & a_{32} & & \\
\vdots & \vdots & \vdots & \ddots & \\
c_s & a_{s1} & a_{s2} & \cdots & a_{s,s-1} \\
\hline
 & b_1 & b_2 & \cdots & b_s \\
 & \hat{b}_1 & \hat{b}_2 & \cdots & \hat{b}_s
\end{array}$$

Table 1: Embedded Runge-Kutta tableau with orders $p(\hat{p})$

where b_i are the p-order weights to obtain y_1 and $\hat{b}_i$ are the $\hat{p}$-order weights to obtain $\hat{y}_1$. The difference $|y_1 - \hat{y}_1|$ provides an estimate of the local error and is used for step size control.

Automatic Step Size Control

The step size control algorithm adjusts h to keep the estimated local error within prescribed tolerances. Given absolute and relative tolerances Atol_i and Rtol_i for each component, we define the scaling factor

$$\mathrm{sc}_i = \mathrm{Atol}_i + \max(|y_{0,i}|, |y_{1,i}|) \cdot \mathrm{Rtol}_i$$

such that the error satisfies componentwise

$$|y_{1i} - \hat{y}_{1i}| \leq sc_i,$$

and compute the normalized error measure

$$err = \sqrt{\frac{1}{n} \sum_{i=1}^{n} \left(\frac{y_{1,i} - \hat{y}_{1,i}}{\mathrm{sc}_i} \right)^2}$$

The value of err is compared against unity to assess whether the current step size is appropriate. Assuming the error behaves asymptotically as $err \approx Ch^{q+1}$ and imposing the condition $1 \approx Ch_{\mathrm{opt}}^{q+1}$, where $q = \min(p, \hat{p})$, we obtain an estimate for the optimal step size as

$$h_{\mathrm{opt}} = h \left(\frac{1}{err} \right)^{1/(q+1)}$$

To ensure robustness, a safety factor fac (typically 0.8 or 0.9) is applied, and the step size change is limited:

$$h_{\mathrm{new}} = h \cdot \min\left(fac_{\max}, \max\left(fac_{\min}, fac \cdot err^{-1/(q+1)}\right)\right)$$

If $err \leq 1$, the computed step is accepted and the solution is advanced with y_1, trying a new step with h_{new}. Otherwise, the step is rejected and computations are repeated with h_{new}.

The maximal step size increase $fac_{\max}$ (typically between 1.5 and 5) prevents excessive enlargements and ensures safety, though small values increase work unnecessarily. Set $fac_{\max} = 1$ after rejections (Shampine & Watts 1979).

When y_1 is lower order than $\hat{y}_1$, $y_1 - \hat{y}_1$ estimates local error. Alternatively, local extrapolation advances with $\hat{y}_1$, using the difference only for step control, as local errors poorly predict global ones due to stability.

Initial Step Size Selection

Rather than requiring the user to specify an initial step size, an automatic procedure estimates a reasonable starting value. The algorithm is based on approximating the local error $\approx Ch^{p+1}|y^{(p+1)}(t_0)|$ using estimates of the first and second derivatives:

1. Compute $f(t_0, y_0)$ and define $d_0 = \|y_0\|$ and $d_1 = \|f(t_0, y_0)\|$
2. Set $h_0 = 0.01 \cdot d_0/d_1$ (or $h_0 = 10^{-6}$ if d_0 or d_1 is smaller than 10^{-5})
3. Perform one explicit Euler step: $y_1 = y_0 + h_0 f(t_0, y_0)$, and evaluate $f(t_0 + h_0, y_1)$
4. Estimate the second derivative: $d_2 = \|f(t_0 + h_0, y_1) - f(t_0, y_0)\|/h_0$

5. Compute h_1 from $h_1^{p+1} \cdot \max(d_1, d_2) = 0.01$. If $\max(d_1, d_2) \leq 10^{-15}$, set $h_1 = \max(10^{-6}, h_0 \cdot 10^{-3})$

6. Select $h = \min(100 \cdot h_0, h_1)$ as the initial step size

This procedure generally yields a reasonable initial step size and prevents extremely poor starting choices, which is crucial for the overall robustness of the adaptive ERK implementation.

2 Implementation

Fixed Step size methods

- **ERK3: Third-Order Runge-Kutta Method**

  For the third-order implementation, we use a 3-stage explicit Runge-Kutta method.

$$\begin{array}{c|ccc} 0 & & & \\ 1/3 & 1/3 & & \\ 2/3 & 0 & 2/3 & \\ \hline & 1/4 & 0 & 3/4 \end{array}$$

Table 2: Butcher tableau: Heun's third-order ERK3 method

- **ERK4: Fourth-Order Runge-Kutta Method**

  For the fourth-order implementation, we use the classical RK4 method.

$$\begin{array}{c|cccc} 0 & & & & \\ 1/3 & 1/3 & & & \\ 2/3 & -1/3 & 1 & & \\ 1 & 1 & -1 & 1 & \\ \hline & 1/8 & 3/8 & 3/8 & 1/8 \end{array}$$

Table 3: Butcher tableau: Classical fourth-order ERK4 method

- **ERK4-5 with Richardson Extrapolation**

  We implemented a fourth-order method with Richardson extrapolation. After computing two steps of size h to reach $t_n + 2h$, we also compute one big step of size $2h$ from t_n. For a fourth-order method ($p = 4$), the extrapolated solution is

$$\hat{y}_{n+2} = y_{n+2} + \frac{y_{n+2} - w}{2^4 - 1} = y_{n+2} + \frac{y_{n+2} - w}{15}$$

  This provides a fifth-order approximation, and the error estimate for y_{n+2} is $(y_{n+2} - w)/15$. While this approach requires additional function evaluations compared to the following embedded formulas, it increases the order of the final approximation from 4 to 5.

Adaptive Step Size Selection

- **BS3(2): Bogacki-Shampine Third/Second-Order Method**

  The Bogacki-Shampine method is a 4-stage embedded Runge-Kutta formula of orders 3 and 2:

$$
\begin{array}{c|cccc}
0 & & & & \\
1/2 & 1/2 & & & \\
3/4 & 0 & 3/4 & & \\
1 & 2/9 & 1/3 & 4/9 & \\
\hline
 & 2/9 & 1/3 & 4/9 & 0 \\
 & 7/24 & 1/4 & 1/3 & 1/8
\end{array}
$$

Table 4: Butcher tableau: Bogacki-Shampine BS3(2) method

- **ERK4(3): Embedded Fourth/Third-Order Runge-Kutta Method (3/8 Rule with FSAL)**

  For the embedded Runge–Kutta implementation of orders 4(3), we utilize the classical 4-stage 3/8 rule of order 4 paired with a third-order embedded estimator. Although it is mathematically impossible to construct a strict 4(3) pair using only $s = 4$ stages (as we will see in the Exercise 2), this limitation is circumvented by incorporating a 5th stage for error estimation via the FSAL (First Same As Last) technique.

  The 5th stage corresponds to $K_5 = f(t_0+h, y_1)$ (the row $a_{5j} = b_j$ $(j = 1, \ldots, 4)$ is set so as to "incorporate" the structure of the principal method) which is computed at the end of the current step and reused as K_1 of the next step $(K_5^{(n)} = K_1^{(n+1)})$ - incurring no extra computational cost - when the step is accepted.

  The embedded weights $\hat{b}_i$ (lower row) are defined by solving the linear system of the 4 order conditions for $p = 3$ (elementary trees up to order 3):

$$
\begin{aligned}
\sum_{i=1}^{5} \hat{b}_i &= 1, && p = 1 \\
\sum_{i=1}^{5} \hat{b}_i c_i &= \frac{1}{2}, && p = 2 \\
\sum_{i=1}^{5} \hat{b}_i c_i^2 &= \frac{1}{3}, && p = 3 \\
\sum_{i=1}^{5} \hat{b}_i \sum_{j=1}^{5} a_{ij} c_j &= \frac{1}{6}, && p = 3
\end{aligned}
$$

  with the specific choices $\hat{b}_4 = 0$ and $\hat{b}_5 = \frac{1}{6}$.

  Solving this system we obtain:

$$
\begin{aligned}
&\hat{b}_1 = 2b_1 - \frac{1}{6} = \frac{1}{12}, \quad \hat{b}_2 = 2(1 - c_2)b_2 = \frac{1}{2}, \\
&\hat{b}_3 = 2(1 - c_3)b_3 = \frac{1}{4}, \quad \hat{b}_4 = 0, \quad \hat{b}_5 = \frac{1}{6}.
\end{aligned}
$$

0					
1/3	1/3				
2/3	-1/3	1			
1	1	-1	1		
	1/8	3/8	3/8	1/8	
	1/12	1/2	1/4	0	1/6

Table 5: Butcher tableau: Embedded ERK4(3) method

- **DP5(4): Dormand-Prince Fifth/Fourth-Order Method**

  The Dormand-Prince method is a 7-stage embedded Runge-Kutta formula of orders 5 and 4 (MATLAB ode45):

0							
1/5	1/5						
3/10	3/40	9/40					
4/5	44/45	-56/15	32/9				
8/9	19372/6561	-25360/2187	64448/6561	-212/729			
1	9017/3168	-355/33	46732/5247	49/176	-5103/18656		
1	35/384	0	500/1113	125/192	-2187/6784	11/84	0
	35/384	0	500/1113	125/192	-2187/6784	11/84	0
	5179/57600	0	7571/16695	393/640	-92097/339200	187/2100	1/40

Table 6: Butcher tableau: Dormand-Prince DP5(4) method

Step Size Control Parameters

For all implementations, we used the following parameters in the step size control algorithm:

- Safety factor: $\text{fac} = 0.8$
- Minimum step size factor: $\text{fac}_{\min} = 0.1$
- Maximum step size factor: $\text{fac}_{\max} = 5.0$ (reduced to 1.0 after a rejected step)
- Error exponent: $\frac{1}{q+1}$ where q is the order of the error estimate

The tolerances Atol and Rtol were varied across test problems to analyze the performance characteristics of each method.

3 Numerical Experiments

Phase 1: Embedded Methods on Multiple Test Cases

To assess the behavior of embedded RK formulas in a general setting, we tested the previous three different tableaux on multiple test problems:

1. The two-dimensional Brusselator system exhibiting stiff-like behavior and oscillations
2. The scalar ODE $y' = 3 + \cos(t) - y$ with initial condition $y(0) = 1$

3. The scalar ODE $y' = y(2-y)(2+y)$ with initial condition $y(0) = 5$

As output we have controlled the evolution of step size h underlining the rejected steps, the analysis of local and global errors and the approximated solution.

Interactive MATLAB Application

An interactive MATLAB application was developed to facilitate systematic testing and parameter exploration. The application allows the user to select:

- The test equation (Brusselator, or either scalar ODE)
- The embedded tableau (ERK4(3), BS3(2), or DP5(4))
- Absolute tolerance (`Atol`) and relative tolerance (`Rtol`)
- Safety factor (`fac`) used in the step size controller

This tool enables exploration of the trade-off between accuracy and efficiency, and visualization of how different embedded pairs respond to changes in error tolerances and adaptive strategy aggressiveness. The application displays the solution trajectory, step size evolution (including accepted and rejected steps), and error behavior throughout the integration.

Results from Multiple Test Cases

The three embedded methods were tested on all three ODEs with various tolerance settings. Key findings include:

- **DP5(4) superiority**: The Dormand-Prince method consistently outperformed lower-order pairs across all test problems, demonstrating excellent efficiency at moderate to high precision requirements
- **Problem-dependent behavior**: The scalar ODEs exhibited smoother step size evolution compared to the Brusselator, with fewer step rejections due to less chaotic dynamics
- **Tolerance sensitivity**: Tighter tolerances revealed the order advantage of DP5(4), while at loose tolerances BS3(2) remained competitive due to its lower per-step cost
- **Safety factor impact**: Smaller safety factors reduced step rejections but increased total computational cost by being more conservative in step size increases

The interactive application proved valuable for understanding the interplay between method order, problem characteristics, and controller parameters in adaptive step size strategies.

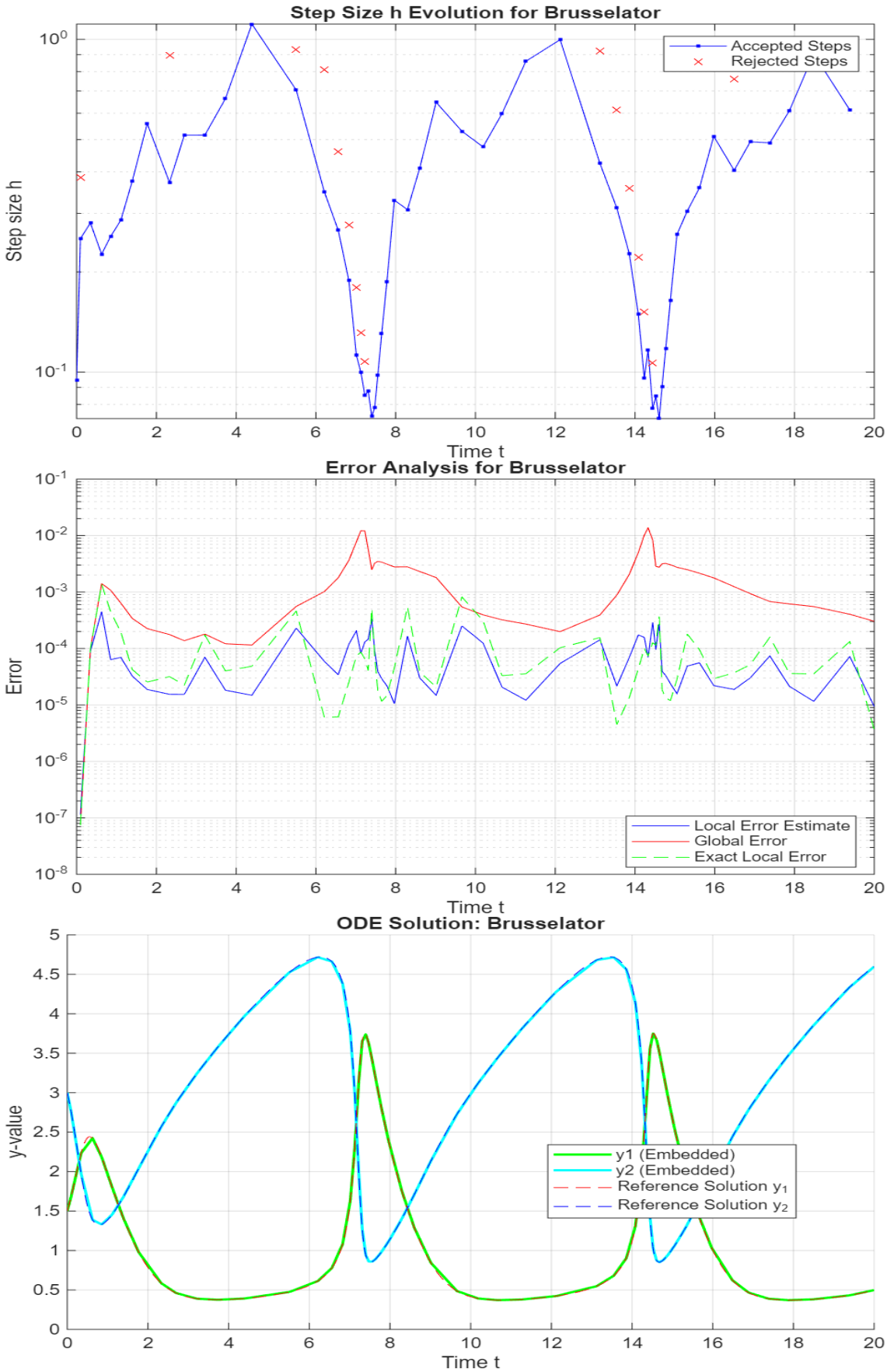

Figure 1: DP5(4) method applied to Brusselator system
(Atol = Rtol = 10^{-4}, $fac = 0.8$)

Phase 2: Detailed Comparison on the Brusselator

In the second phase of this study, we perform a work-precision analysis to compare the computational efficiency of adaptive and fixed-step explicit Runge-Kutta (ERK) methods applied to the Brusselator problem. The integration is performed over the interval $t \in [0, 20]$ with initial conditions $y_0 = [1.5, 3]^T$.
To establish a ground truth for global error calculations, a highly accurate baseline solution is first computed using MATLAB's `ode45` solver with stringent tolerances (RelTol $= 10^{-12}$, AbsTol $= 10^{-14}$).
We evaluate three embedded RK pairs—BS3(2), ERK4(3), and DP5(4)—using a custom adaptive step-size solver across a range of tolerances (10^{-4} to 10^{-10}). These are compared against three non-adaptive strategies—ERK3, ERK4, and ERK4 enhanced with Richardson extrapolation—evaluated across a predefined set of integration steps ($N \in \{100, 200, \ldots, 1500\}$).
The results are synthesized in the work-precision diagram shown in Figure 2, which plots the global error against the total number of function evaluations on a log-log scale.

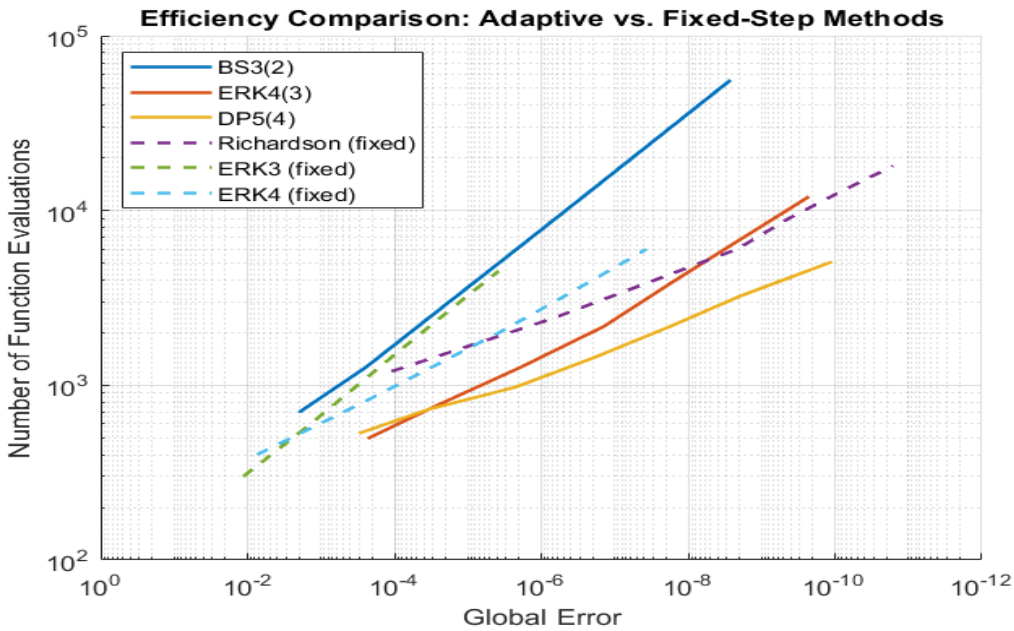

Figure 2: Efficiency Comparison: Work-precision diagram showing the trade-off between the number of function evaluations and global error for adaptive and fixed-step ERK methods.

Analysis of the Work-Precision Diagram

With the x-axis reversed, the most efficient methods are those occupying the lower-right region of the graph, as they achieve lower errors with fewer function evaluations. The diagram highlights several key insights:

- **Order of Convergence:** The slope of each line is proportional to $-1/p$, where p is the order of the method. The third-order methods (BS3(2) and ERK3) exhibit visibly steeper slopes, indicating that their computational cost grows rapidly as tolerances tighten. The fourth and fifth-order methods display flatter slopes, demonstrating better scalability for high-accuracy requirements.

- **Adaptive vs. Fixed-Step Overhead:** A critical observation is that the fixed-step ERK3 is more efficient than the adaptive BS3(2) across the entire tested range. This suggests that for the Brusselator system, the step-size adaptation overhead and the cost of rejected steps in BS3(2) heavily penalize its performance, negating the theoretical benefits of adaptivity for this specific order. Conversely, the adaptive ERK4(3) scales better than the fixed ERK4 at stricter tolerances (errors below 10^{-5}).

- **Richardson Extrapolation:** The Richardson-enhanced method successfully achieves a flatter slope, consistent with a fifth-order approximation. However, because it significantly inflates the baseline cost (requiring 12 evaluations per step), its efficiency curve remains consistently above DP5(4), making it less competitive overall.

- **Optimal Method Selection:** Dormand-Prince DP5(4) emerges as the undisputed optimal choice for this problem. For any global error target below 10^{-4}, it achieves the required accuracy with significantly fewer function evaluations than all other tested implementations.

Detailed Performance Snapshot

To complement the overall efficiency curves, Table 7 provides a detailed snapshot of the methods' internal behavior for a specific integration scenario. Note that the non-adaptive methods were executed with a fixed budget of $N = 600$ steps, while the adaptive methods were run at a specific tolerance level to illustrate step-size variations.

Method	h_{min}	h_{max}	FEvals	Error	Total Steps	Rejected Steps
ERK3	N/A	N/A	1800	6.22×10^{-5}	600	0
ERK4	N/A	N/A	2400	1.71×10^{-6}	600	0
Richardson	N/A	N/A	6600	1.10×10^{-9}	600	0
BS3(2)	2.16×10^{-3}	7.41×10^{-2}	4192	2.70×10^{-6}	1391	4
ERK4(3)	6.05×10^{-3}	1.42×10^{-1}	1630	1.40×10^{-7}	537	4
DP5(4)	1.90×10^{-2}	3.32×10^{-1}	1221	2.05×10^{-7}	189	12

Table 7: Performance metrics and step-size dynamics for fixed-step ($N = 600$) and adaptive methods on the Brusselator problem.

This snapshot reveals the mechanics behind the efficiency of higher-order adaptive solvers. While DP5(4) experiences the highest number of rejected steps (12), its ability to safely take significantly larger steps ($h_{max} = 3.32 \times 10^{-1}$) allows it to traverse the integration interval in just 189 total steps. This drastically reduces the total function evaluations compared to BS3(2), which requires 1391 steps and yields an error an order of magnitude larger. Furthermore, when evaluating the fixed-step strategies restricted to a 600-step budget, the advantage of adaptivity becomes clear. DP5(4) achieves better accuracy (2.05×10^{-7}) than the fixed ERK4 (1.71×10^{-6}) using approximately half the function evaluations (1221 versus 2400). Similarly, the adaptive ERK4(3) outperforms its fixed-step counterpart in both computational cost and precision. Although Richardson extrapolation yields the lowest absolute error (1.10×10^{-9}), its rigid step size imposes a massive burden of 6600 evaluations, illustrating the heavy computational penalty of non-adaptive, high-order approaches.

Conclusion

The comparative analysis of explicit Runge-Kutta methods yields several key findings regarding step-size control strategies:

- **Optimal Method:** The Dormand-Prince DP5(4) embedded pair is the undisputed most efficient solver among those tested. It consistently achieves high accuracy with significantly fewer function evaluations, particularly at stringent tolerances.
- **The Cost of Adaptivity:** Automatic step-size control introduces computational overhead that is not always justifiable. For lower-order methods applied to stiff-like problems (e.g., BS3(2) on the Brusselator), the cost of rejected steps and step-size adaptation makes its fixed-step equivalent (ERK3) more efficient.
- **High-Order Advantage:** For fourth-order methods and above, the benefits of adaptivity easily outweigh the overhead. The adaptive ERK4(3) strictly outperforms its fixed-step ERK4 counterpart in both computational cost and precision.
- **Inefficiency of Richardson Extrapolation:** While Richardson extrapolation successfully elevates a fourth-order method to fifth-order accuracy, its rigid step size and massive baseline cost render it uncompetitive compared to high-order embedded adaptive techniques like DP5(4).
- **Problem Dependence:** The efficiency of adaptive strategies is highly sensitive to the problem's dynamics. Smooth systems allow for aggressive step-size increases, whereas chaotic or stiff-like systems induce frequent step rejections, demanding robust controller tuning.

Ultimately, while fixed-step methods can be viable for low-order, low-precision applications, pairing high-order embedded formulas like DP5(4) with adaptive step-size control remains the most robust and computationally efficient approach for general initial value problems.

4 Exercises

A Show that Runge's method can be interpreted as two Euler steps (with step size h/2), followed by a Richardson extrapolation.

B Prove that no 4-stage Runge-Kutta method of order 4 admits an embedded formula of order 3.

C Show that the step size strategy is invariant with respect to a rescaling of the independent variable. This means that it produces equivalent step size sequences when applied to the two problems:

$$y' = f(t, y), \quad y(0) = y_0, \quad y(t_{\text{end}}) = ?,$$

and

$$z' = \sigma f(\sigma t, z), \quad z(0) = y_0, \quad z\left(\frac{t_{\text{end}}}{\sigma}\right) = ?,$$

with initial step sizes h_0 and h_0/σ, respectively.

Solutions

Solution A

Consider Euler's method with full step size h for the initial value problem

$$y'(t) = f(t, y), \quad y(t_n) = y_n.$$

A single Euler step gives

$$y_{n+1} = y_n + hf(t_n, y_n).$$

If instead we use two half steps of size $h/2$, we obtain

$$y_{n+\frac{1}{2}} = y_n + \frac{h}{2} f(t_n, y_n),$$

$$y_{n+1} = y_{n+\frac{1}{2}} + \frac{h}{2} f\left(t_n + \frac{h}{2}, y_{n+\frac{1}{2}}\right),$$

so that

$$y_{n+1} = y_n + \frac{h}{2}\left[f(t_n, y_n) + f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2} f(t_n, y_n)\right)\right].$$

Let w denote the result of a single Euler step of size h,

$$w = y_n + hf(t_n, y_n),$$

which is a first-order method ($p = 1$). Applying Richardson extrapolation with

$$\hat{y}_{n+1} = y_{n+1} + \frac{y_{n+1} - w}{2^p - 1}$$

yields, for $p = 1$,

$$\hat{y}_{n+1} = 2y_{n+1} - w = y_n + hf\left(t_n + \frac{h}{2}, y_n + \frac{h}{2} f(t_n, y_n)\right),$$

which is exactly Runge's two-stage second-order method.

Solution B

By contradiction, let us consider a 4-stage Runge–Kutta method of order four and denote by b_j the weights of the primary method and by $\hat{b}_j$ those of a embedded third-order formula. Define $d_j = b_j - \hat{b}_j$ for $j = 1, \ldots, 4$. The aim is to show that $d_j = 0$ for all j, which would imply $b_j = \hat{b}_j$ and thus contradict the assumption that the embedded formula has strictly lower order.

Introduce the matrices

$$U = \begin{pmatrix} b_2 & b_3 & b_4 \\ b_2 c_2 & b_3 c_3 & b_4 c_4 \\ d_2 & d_3 & d_4 \end{pmatrix}, \quad V = \begin{pmatrix} c_2 & c_2^2 & \sum_j a_{2j} c_j - \frac{c_2^2}{2} \\ c_3 & c_3^2 & \sum_j a_{3j} c_j - \frac{c_3^2}{2} \\ c_4 & c_4^2 & \sum_j a_{4j} c_j - \frac{c_4^2}{2} \end{pmatrix},$$

where the entries of V are built from the Butcher coefficients (a_{ij}, c_i). Using the order conditions up to order four, one shows that

$$UV = \begin{pmatrix} \frac{1}{2} & \frac{1}{3} & 0 \\ \frac{1}{3} & \frac{1}{4} & 0 \\ 0 & 0 & 0 \end{pmatrix},$$

whose upper 2×2 block has nonzero determinant. A linear algebra lemma then implies either $Ve_3 = 0$ or $U^\top e_3 = 0$, where $e_3 = (0, 0, 1)^\top$. The first alternative contradicts the structure of the Butcher tableau (since $c_i \neq 0$ for $i \geq 2$), so necessarily $U^\top e_3 = 0$, which yields $d_2 = d_3 = d_4 = 0$.

Finally, using the consistency condition $\sum_{j=1}^4 b_j = \sum_{j=1}^4 \hat{b}_j = 1$, we obtain

$$\sum_{j=1}^{4} d_j = 0 \quad \Rightarrow \quad d_1 = 0,$$

and therefore $b_j = \hat{b}_j$ for all j. This proves that a 4-stage order-4 Runge–Kutta method cannot admit an embedded order-3 formula.

Solution C

The adaptive step-size strategy determines the sequence of step sizes h_k from the normalized error estimate

$$err = \sqrt{\frac{1}{n} \sum_{i=1}^{n} \left(\frac{y_{1,i} - \hat{y}_{1,i}}{\mathrm{sc}_i} \right)^2},$$

and updates the step length according to

$$h_{\text{new}} = h \left(\frac{1}{err} \right)^{1/(q+1)},$$

possibly modified by safety factors and bounds.

Consider the two problems

$$y' = f(t, y), \quad y(0) = y_0,$$

and

$$z' = \sigma f(\sigma t, z), \quad z(0) = y_0,$$

with initial step sizes h_0 and h_0/σ, respectively. For an embedded method of order p, the local error behaves as

$$\|y_k - \hat{y}_k\| \approx C h^{p+1} |y^{(p+1)}(0)|, \quad \|z_k - \hat{z}_k\| \approx \tilde{C} \left(\frac{h}{\sigma} \right)^{p+1} |z^{(p+1)}(0)|.$$

Using the relation $z^{(p+1)}(0) = \sigma^{p+1} y^{(p+1)}(0)$, one obtains

$$\|z_k - \hat{z}_k\| \approx \tilde{C} h^{p+1} |y^{(p+1)}(0)|,$$

so that the normalized errors for the two problems are asymptotically identical at each step. Since the step-size update depends only on err, both problems generate the same sequence of step sizes, up to the initial rescaling of the independent variable. This shows that the step-size control strategy is invariant under time rescaling.

If we replace err by err/h and q by $(q-1)$ (*error per unit step*) we obtain:

$$\frac{\|y_k - \hat{y}_k\|}{h} \approx C h^q |y^{(q+1)}(0)|$$

$$\frac{\|z_k - \hat{z}_k\|}{h} \approx \tilde{C} \left(\frac{h}{\sigma} \right)^q \sigma^{q+1} |y^{(q+1)}(0)| = \tilde{C} h^q \sigma |y^{(q+1)}(0)|$$

So the errors exhibit a different behavior, due to the presence of the factor σ, which persists in the error expression of the rescaled problem.